

Desarrollo de Gestión y Consulta de Tickets

Actividad N°: 21 **Fecha:** 29/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar y documentar los endpoints de consulta de boletos, diferenciando el acceso general (Jefe) del acceso restringido por punto de trabajo (Staff).

2. Endpoints de consulta implementados

Endpoint	Acceso	Descripción
GET /api/tickets	jefe, staff	Consulta general con filtros; si es staff, se fuerza el filtro por su puntoTrabajo
GET /api/puntos-venta/:id/tickets	Cualquier autenticado	Tickets asociados a un punto de venta específico (para Jefe)
GET /api/puntos-venta/staff/tickets	staff, impresor	Tickets del punto de trabajo del usuario autenticado
GET /api/puntos-venta/:id/tickets/check-changes	Cualquier autenticado	Verificación ligera de cambios recientes (sincronización)
GET /api/puntos-venta/staff/tickets/check-changes	staff, impresor	Igual al anterior, restringido al punto de trabajo del staff
GET /api/puntos-venta/:id/estadisticas	Cualquier autenticado	Conteo de tickets por localidad dentro de un punto de venta

3. Regla de negocio central: aislamiento por punto de trabajo

Para un usuario Staff, el sistema **nunca** confía en un valor de punto de trabajo enviado desde el cliente: siempre se resuelve del lado del servidor a partir de **req.user.puntoTrabajo**, buscando el **PuntoVenta** correspondiente por nombre y usando sus localidades asociadas para construir el filtro de consulta. Esto asegura que un Staff no pueda ver boletos de otro punto de trabajo aunque manipule la petición HTTP.

4. Corrección aplicada: eliminación de fallback hardcodeado

Durante el desarrollo se identificó que **getTicketsForStaff** contenía un **mapeo de respaldo hardcodeado** (**puntoTrabajoLocalidades**) con nombres de localidades de un evento anterior del proyecto (ej. "Hips Don't Lie PLATINUM", "SOLTERA FAN ZONE"), que se activaba silenciosamente si el nombre del punto de trabajo del usuario no coincidía con ningún **PuntoVenta** real y vigente. Esto contradecía el principio de diseño de **localidades dinámicas** (Semana 2, RF-10) y podía mostrarle a un Staff mal configurado datos de un concierto distinto al actual, sin ninguna advertencia.

Archivo modificado: backend/src/controllers/puntoVentaController.js

```
- let localidadesAsignadas = [];  
-  
- if (puntoVenta) {  
-   localidadesAsignadas = puntoVenta.localidades;  
- } else {  
-   const puntoTrabajoLocalidades = {  
-     'boletería norte': ['GENERAL', 'PREFERENCIA'],  
-     'boletería sur': ['TRIBUNA', 'PALCO'],  
-     'centro comercial': ['Las Mujeres Facturan BOX', 'Antología GOLDEN'],  
-     'punto central': ['Hips Don\'t Lie PLATINUM', 'SOLTERA FAN ZONE'],  
-     'entrada principal': ['GENERAL', 'PREFERENCIA', 'TRIBUNA']  
-   };  
-   localidadesAsignadas = puntoTrabajoLocalidades[userPuntoTrabajo] ||  
+ ['GENERAL'];  
- }  
+ if (!puntoVenta) {  
+   return res.status(404).json({  
+     success: false,  
+     message: `No se encontró un punto de venta activo llamado  
"${userPuntoTrabajo}". Verifique la configuración del punto de trabajo del  
usuario.`  
+   });  
+ }  
+  
+ const localidadesAsignadas = puntoVenta.localidades;
```

Con este cambio, si el punto de trabajo de un usuario Staff está mal configurado (no coincide con ningún Punto de Venta activo), el sistema responde con un error explícito en vez de mostrar datos de un evento incorrecto.

5. Consideraciones de rendimiento aplicadas

- Uso de `.lean()` en consultas de solo lectura para reducir el overhead de hidratar documentos Mongoose completos.
- `maxTimeMS()` en todas las consultas de listado y conteo, para evitar que una consulta lenta bloquee el servidor indefinidamente.
- Paginación con límite máximo forzado de 100 elementos por página, sin importar lo que solicite el cliente.
- Ejecución en paralelo (`Promise.all`) de la consulta de datos y el conteo total, en vez de secuencial.

6. Conclusiones del día

Se implementaron los endpoints de consulta diferenciando correctamente el alcance de datos según el rol, y se corrigió una inconsistencia real de diseño (fallback hardcodeado de un evento anterior) que violaba el principio de localidades dinámicas acordado desde la Semana 2.

Observaciones: Corrección aplicada y verificada; sin observaciones adicionales.